



Universidad
Tecnológica
del Perú

Universidad Tecnológica del Perú

Guía De Laboratorio 08

Sistema Inteligente de Apoyo Pedagógico y Gestión del Aprendizaje para Instituciones Educativas

INTEGRANTES – GRUPO 6:

Rivera Castañeda, Jesús Martín	- U21213646
Quispe Fernández, Alonso	- U20208103
Alanya Guevara, Ben Andre	- U21315933
Ninatanta Fernández, Renato Ventura	- U21205210

ASIGNATURA:

Curso Integrador II: Software

DOCENTE:

Barturen Huaman, Renzo Renato

SECCIÓN:

31738

Lima – Perú

ÍNDICE

Índice	2
1. Resultado y alcance del incremento	3
1.1 Producto mínimo de la semana	3
2. Correspondencia con el sílabo.....	3
3. Fundamentos de prueba aplicados.....	4
4. Arquitectura del incremento.....	4
5. Casos observables	4
6. Pruebas funcionales de la API con MySQL real	5
7. Pruebas de interfaz con Playwright y axe.....	6
8. Carga ligera con k6	6
9. Empaquetado y despliegue con Docker y Caddy	6
10. Publicación en VM cloud.....	7
11. Persistencia y reversión	7
12. Riesgos conectados al incremento.....	8
13. Entregables y estado de evidencia (E0–E11).....	8
14. Checklist de cierre	9

1. Resultado y alcance del incremento

Este avance corresponde a la Semana 8 del Curso Integrador II: Software y da continuidad al incremento de catálogo (Semana 6) y al flujo autenticado de envío de un intento de ejercicio (Semana 7) sobre ESTUD-IA. El objetivo de esta etapa es comprobar que la aplicación construida cumple sus requerimientos funcionales y no funcionales, y desplegarla fuera del entorno de desarrollo: el mismo frontend React, la misma API Laravel y la misma base MySQL, sin crear otra aplicación ni reemplazar los módulos existentes.

El resultado es una versión de laboratorio con pruebas reproducibles, una métrica inicial de carga y un despliegue verificado localmente y en una máquina virtual cloud autorizada.

1.1 Producto mínimo de la semana

- Pruebas de integración de la API (PHPUnit) contra un MySQL real de pruebas, no contra mocks.
- Pruebas end-to-end con Playwright sobre React real (catálogo, login, envío de un intento).
- Auditoría de accesibilidad con axe sobre las rutas /cursos e /ingresar.
- Carga ligera con k6 sobre el endpoint de catálogo/contenidos.
- Imágenes Docker del backend (Laravel) y del frontend (React + Caddy), orquestadas con Docker Compose.
- Despliegue verificado en una VM cloud con HTTPS automático (Caddy) y comprobación de persistencia en MySQL tras reiniciar el contenedor de la API.

2. Correspondencia con el sílabo

La siguiente tabla adapta la correspondencia de la guía original (Angular + Spring Boot + PostgreSQL + Testcontainers) a las tecnologías reales de ESTUD-IA (React + Laravel + MySQL).

Tema de la semana 8	Aplicación en ESTUD-IA	Evidencia
Validación del servicio y pruebas funcionales/no funcionales	Comprobar catálogo, login, autorización, envío de intento, errores, concurrencia, accesibilidad y latencia.	Matriz de casos API0x/WEB0x (sección 5) con resultado real.
Pruebas de integración contra base real	PHPUnit Feature tests contra un MySQL de pruebas real (base dedicada o efímera), no contra un repositorio simulado.	Reporte de phpunit con los casos nuevos aprobados.
Pruebas en navegador sin simular la API	Playwright contra React real, con la API Laravel y MySQL activos (no fixtures estáticos).	Informe HTML de Playwright.
Calidad no funcional	k6 contra /api/v1/contenidos; axe sobre /cursos e /ingresar; revisión manual de teclado y zoom 200 %.	Resumen k6 e informe axe.
Enfoques y mejores prácticas de despliegue	Imágenes Docker (PHP-FPM/Laravel y React+Caddy), Docker Compose en una VM, HTTPS automático con Caddy.	compose.yaml, Dockerfiles, Caddyfile y smoke test.
Pruebas en plataforma cloud	Publicar la misma versión en una VM cloud autorizada y repetir smoke test y Playwright contra la URL HTTPS pública.	Capturas/; registro de IDs de imagen y URL.

3. Fundamentos de prueba aplicados

Se mantienen, sin cambios de criterio, los conceptos de la guía original: verificar (contrastar contra una condición especificada, por ejemplo, un Alumno autenticado envía un intento y recibe HTTP 201) y validar (comprobar que la solución permite cumplir la necesidad real de uso). Una prueba funcional pregunta qué hace la aplicación; una prueba no funcional pregunta en qué condiciones lo hace.

Nivel	Qué ejecuta en ESTUD-IA	Límite de su evidencia
Unitario	Una clase o Service con colaboradores controlados (por ejemplo <code>IntentoEjercicioServiceTest</code> con mocks).	No prueba SQL, migraciones, Sanctum ni la red real.
Integración de API	Laravel real, middleware, Sanctum y MySQL de pruebas real (sección 6).	No ejecuta React ni la configuración cloud.
Extremo a extremo	Navegador, React y API reales mediante Playwright, local o cloud (sección 7).	Mayor costo; depende de una agenda/catálogo sintético controlado.
Smoke de despliegue	Un conjunto pequeño de comprobaciones críticas de la URL publicada.	No sustituye la regresión completa ni una auditoría de seguridad.

4. Arquitectura del incremento

Se agregan tres capacidades sobre la arquitectura de las Semanas 6 y 7, sin introducir un nuevo servicio de negocio: pruebas de integración, pruebas end-to-end/carga, y empaquetado para despliegue.

Componente	Responsabilidad en esta semana	No debe hacer
PHPUnit (Feature tests)	Ejercitar los endpoints reales (catálogo, login, envío de intento) contra MySQL, usando <code>RefreshDatabase</code> o una base de pruebas dedicada.	Simular el Service o el Repository con un doble; eso pertenece al nivel unitario.
Playwright + axe	Verificar el comportamiento visible del Alumno en React real y auditar accesibilidad de <code>/cursos</code> e <code>/ingresar</code> .	Sustituir la verificación de la API; no reemplaza las pruebas de integración.
k6	Medir latencia (p95) y tasa de error del catálogo bajo una carga breve y controlada.	Ejercitar login o escritura; el script solo hace lecturas, igual que en la guía.
Docker + Compose	Empaquetar backend (Laravel) y frontend (React) como imágenes reproducibles, orquestadas junto a MySQL.	Exponer MySQL ni el puerto interno de la API al host o a Internet.
Caddy	Servir el build de React, enrutar <code>/api</code> hacia Laravel y terminar HTTPS automático en la VM cloud.	Publicar rutas de diagnóstico internas (equivalente a <code>/actuator</code>) fuera de <code>/healthz</code> .

5. Casos observables

La matriz retoma el catálogo de la Semana 6 y el flujo autenticado de la Semana 7 (iniciar sesión → enviar intento de ejercicio), con los mismos criterios de la guía original.

Caso	Precondición y acción	Resultado esperado
API01	Base de pruebas aislada; GET /api/v1/cursos y /api/v1/contenidos para MAT101 el 03/09/2026.	200; contenidos del fixture de la Semana 6.
API02	POST intentos de ejercicio sin token Sanctum.	401 y ningún intento nuevo.
API03	Login con cuenta sintética válida y otra con contraseña incorrecta.	200 con token utilizable; intento inválido 401 genérico.
API04	Token de Docente en POST intentos de ejercicio.	403 y ningún intento nuevo.
API05	Token de Alumno y ejerciciold con formato inválido.	422 y ningún intento nuevo.
API06	Alumno envía un intento y repite el mismo POST.	201 y luego 409; un solo registro y un evento de auditoría.
API07	Dos solicitudes concurrentes por el mismo ejercicio.	Un 201 y un 409; una sola fila, por el bloqueo de fila de la Semana 7.
API08	Token modificado, expirado o inexistente en personal_access_tokens.	401 y ningún intento nuevo.
WEB01	Buscar contenidos desde React real.	Cantidad renderizada coherente con la respuesta de la API.
WEB02	Abrir una ruta protegida de envío de intento sin sesión.	Redirección a /ingresar preservando returnUrl.
WEB03	Buscar un curso/fecha sin contenidos publicados.	Estado vacío con indicación para cambiar filtros.
WEB04	Ingreso y un envío de intento controlado sobre un ejercicio libre; recargar.	201, confirmación visible y sesión perdida al recargar (token solo en memoria).

6. Pruebas funcionales de la API con MySQL real

En el ecosistema Laravel/PHP es una base de datos MySQL real y aislada para pruebas combinada con el trait RefreshDatabase, que aplica las migraciones y limpia el estado entre pruebas, igual intención que @ServiceConnection + Testcontainers.

Concepto en la guía (Spring Boot)	Equivalente en Laravel	Aplicación en ESTUD-IA
@SpringBootTest(webEnvironment = RANDOM_PORT) + Testcontainers PostgreSQL	Feature test de Laravel (extends TestCase) con RefreshDatabase, contra una conexión MySQL de pruebas real configurada en phpunit.xml	FunctionalApiTest.php ejercita rutas reales, no un controller aislado
PasswordEncoder + TokenService para crear usuarios y emitir JWT de prueba	Hash::make() para la contraseña y \$user->createToken() de Sanctum para emitir el token de prueba	Los usuarios de prueba (Alumno, Docente) se crean igual que el DemoUserSeeder de la Semana 7

Concepto en la guía (Spring Boot)	Equivalente en Laravel	Aplicación en ESTUD-IA
@BeforeEach limpia únicamente las tablas del contenedor efímero	RefreshDatabase limpia (o migra de nuevo) la base de pruebas entre cada test automáticamente	Ningún caso de prueba toca la base de desarrollo ni la base de despliegue
mvn -Dtest=FunctionalApiTest test / mvn verify	php artisan test -- filter=FunctionalApiTest / php artisan test	Debe ejecutarse con Docker/MySQL de pruebas activo; si no está disponible, el caso queda Bloqueado, no aprobado por omisión

7. Pruebas de interfaz con Playwright y axe

Playwright y axe no dependen del framework de backend, así que se usan exactamente igual que en la guía original, apuntando a la SPA de React real (no a Angular) y a la API Laravel real (no simulada).

- Dependencias de desarrollo: @playwright/test y @axe-core/playwright, con versiones exactas fijadas en package-lock.json, igual criterio que la guía.
- Los casos de lectura (equivalentes a WEB01-WEB03) se ejecutan repetidamente contra la base de pruebas; el caso de escritura (WEB04, envío de un intento real) se reserva para un ejercicio sintético libre y se ejecuta una sola vez, igual que F04 en la guía.
- Los localizadores usan getByRole y getByLabel sobre la semántica visible de React, igual criterio de estabilidad y accesibilidad que en Angular.

8. Carga ligera con k6

El script de carga se mantiene conceptualmente igual: solo hace lecturas (GET /api/v1/contenidos), no ejercita login ni escritura, y mide p95 y tasa de error con los mismos umbrales didácticos de la guía (5 usuarios virtuales durante 1 minuto; p95 < 500 ms; errores < 1 %), ejecutado contra el despliegue local empaquetado (sección 9), no contra el servidor de desarrollo (npm run dev / php artisan serve).

La única adaptación es la URL base, que apunta al API Gateway de Laravel detrás de Caddy en lugar del backend Spring Boot; la lógica de medición, los thresholds y la interpretación de resultados (p95 como percentil, no promedio; una sola ejecución no certifica capacidad) no cambian.

9. Empaquetado y despliegue con Docker y Caddy

La relación de componentes de la guía se traduce directamente al stack de ESTUD-IA, manteniendo el mismo patrón de red en capas (edge, aplicación, datos).

Acceso	Destino y responsabilidad en ESTUD-IA
Navegador → web	Caddy sirve el build de producción de React (npm run build) y termina HTTPS automático en cloud.
web → api	/api conserva la ruta completa y se reenvía al contenedor de Laravel (PHP-FPM + Nginx o el servidor integrado) por la red interna.
api → db	Eloquent y las migraciones de Laravel acceden a MySQL por una red de datos interna, sin publicar el puerto 3306 al host.

Acceso	Destino y responsabilidad en ESTUD-IA
Salud	Una ruta <code>/healthz</code> propia (o <code>/up</code> , disponible de forma nativa desde Laravel 11) comprueba disponibilidad de la API y la conexión a MySQL.
Persistencia	Un volumen <code>db_data</code> conserva las tablas de MySQL; otro, <code>caddy_data</code> , conserva certificados y claves TLS.

Las imágenes se construyen en dos etapas, igual criterio que la guía: la imagen final del backend no necesita Composer con dependencias de desarrollo; la imagen final del frontend no necesita Node, solo sirve los archivos estáticos generados por `npm run build`. Docker Compose arranca primero la base de datos saludable, después la API y después el frontend; ni MySQL ni el puerto interno de Laravel se publican al host, solo el puerto de Caddy.

El rol de aplicación de la base de datos conserva los mismos principios que la guía: no es un usuario administrador completo, pero sí propietario del esquema para aplicar las migraciones de Laravel durante el laboratorio. En producción corresponde separar la ejecución de migraciones de la ejecución de la aplicación y reemplazar las cuentas de demostración.

10. Publicación en VM cloud

El procedimiento de transferencia y arranque es el mismo que en la guía: se construyen las imágenes localmente, se exportan con `docker save`, se transfieren a la VM autorizada y se arrancan con `--no-build` para evitar una compilación distinta en el servidor. Caddy obtiene y renueva el certificado TLS de forma automática cuando el DNS del dominio de laboratorio apunta correctamente a la VM y los puertos 80/443 están disponibles.

- Las reglas de red del proveedor cloud deben permitir solo SSH desde una IP autorizada, y 80/443 para el tráfico web; el puerto de MySQL y el puerto interno de Laravel no se exponen externamente, igual criterio que PostgreSQL y Spring Boot en la guía.
- Tras publicar, se repiten el smoke test y los casos de lectura de Playwright contra la URL HTTPS pública, y WEB04 se repite una sola vez contra un ejercicio libre de la base cloud (independiente de la base local).
- Se verifica que la navegación profunda (recargar una ruta de React) cargue la SPA y no un 404, y que `/api` responda JSON y no el `index.html` de React, señal de una regla de proxy incorrecta en el Caddyfile.

11. Persistencia y reversión

Se confirma que reiniciar únicamente el contenedor de la API (Laravel) conserva los registros en MySQL: se registra el conteo de intentos de ejercicio y el código de un intento sintético, se reinicia el contenedor de la API, se espera a que la ruta de salud vuelva a estar disponible, y se compara el mismo conteo y código. El respaldo mínimo antes de actualizar usa `mysqldump` en lugar de `pg_dump`, con el mismo criterio de verificación (un archivo no vacío y un volcado legible no garantizan por sí solos una restauración completa, pero sí confirman que existe un respaldo interpretable).

La reversión de versión (volver de una imagen nueva a la anterior) sigue siendo válida solo si el esquema de MySQL de la versión nueva es compatible con la imagen anterior; igual que con Flyway en la guía, no se edita una migración de Laravel ya aplicada ni se ejecuta un comando que borre el volumen de datos para ensayar el rollback.

12. Riesgos conectados al incremento

Se retoman los riesgos R-01 a R-14 registrados en los avances anteriores y se agregan dos riesgos propios de esta semana, originados en la diferencia entre el entorno de referencia de la guía (Testcontainers + PostgreSQL + VM con Spring Boot) y el entorno real del proyecto (MySQL real de pruebas + VM con Laravel).

ID	Riesgo formulado	Disparador	Control / respuesta	Indicador relacionado
R-15	Una prueba de integración que depende de un MySQL de pruebas compartido (en vez de un contenedor efímero por clase) podría arrastrar estado entre ejecuciones y dar falsos aprobados o falsos fallos.	Una prueba pasa de forma aislada pero falla al ejecutar la suite completa, o viceversa.	Usar RefreshDatabase en cada Feature test y, si el equipo lo adopta, un contenedor MySQL efímero por ejecución de CI equivalente a Testcontainers.	DEL-01
R-16	Una regla de proxy del Caddyfile adaptada de Spring Boot a Laravel podría reenviar /api de forma incompleta y exponer el index.html de React en vez de JSON de error.	Una ruta de la API responde HTML con código 200 en vez de JSON.	Verificar explícitamente con curl que /api/* conserva la ruta completa antes del fallback de la SPA, como parte del smoke test.	UX-01

13. Entregables y estado de evidencia (E0–E11)

ID	Entregable	Contenido mínimo	Estado
E0	Inicio controlado	Issue, rama feature/semana8-pruebas-despliegue, commit base.	Pendiente
E1	Matriz de casos	API01–API08 y WEB01–WEB04 con resultado real.	Completado (propuesta)
E2	Regresión existente	Suite de React y PHPUnit sin regresiones antes de agregar pruebas nuevas.	Pendiente
E3	Pruebas de integración de la API	FunctionalApiTest.php contra MySQL real; ocho casos aprobados.	Pendiente
E4	Pruebas end-to-end	Informe Playwright de catálogo, login y envío de intento.	Pendiente
E5	Calidad no funcional	Resumen k6 e informe axe de /cursos e /ingresar.	Pendiente
E6	Empaquetado local	Imágenes Docker, compose.yaml y smoke test aprobados en local.	Pendiente
E7	Publicación cloud	URL HTTPS pública, IDs de imagen y smoke test aprobado.	Pendiente
E8	Persistencia	Mismo conteo e intento antes y después de reiniciar la API.	Pendiente
E9	Respaldo y reversión	Volcado mysqldump legible; reversión ensayada o pendiente justificada.	Pendiente

ID	Entregable	Contenido mínimo	Estado
E10	Control técnico	Salida de pruebas, build y consola sin errores, sin secretos.	Pendiente
E11	Defensa	Presentación y Pull Request con revisión por pares.	Pendiente

14. Checklist de cierre

- Las pruebas de integración de la API corren contra un MySQL real de pruebas, no contra un repositorio simulado.
- Los ocho casos API0x y los cuatro casos WEB0x están documentados con su resultado real, no solo con el esperado.
- Playwright y axe se ejecutan contra React y Laravel reales, en local y luego contra la URL cloud.
- El script k6 solo hace lecturas y usa los mismos umbrales didácticos que la guía, sin forzarlos para ocultar un fallo.
- Las imágenes Docker no exponen MySQL ni el puerto interno de Laravel al host ni a Internet.
- La VM cloud sirve la aplicación por HTTPS con certificado válido, obtenido automáticamente por Caddy.
- Un reinicio del contenedor de la API conserva los datos en MySQL; existe un respaldo mysqldump legible.
- Los riesgos R-15 y R-16 quedan vinculados al registro de riesgos existente de ESTUD-IA.
- Ningún secreto (contraseñas, tokens, cadenas de conexión) aparece en commits, capturas o este documento.